



VAASAN AMMATTIKORKEAKOULU
UNIVERSITY OF APPLIED SCIENCES

Ke Qiu

PERSONAL INTELLIGENT ASSISTANT BASED ON LARGE LANGUAGE MODEL

Personalized Knowledge Extraction and Query Answering Using

Local Data and Large Language Model

School of Technology
2024

ACKNOWLEDGEMENTS

First and foremost, I sincerely thank my supervisor, Magnus Sundell, for his patient guidance, numerous meetings, and detailed feedback, which provided me with invaluable insights and suggestions that were crucial for the completion of this thesis.

Secondly, I would like to express my gratitude to my teacher, Johan Dams. Initially, I had doubts about choosing this topic, but through our email correspondence, he gave me tremendous confidence, and ultimately, I settled on this topic.

Lastly, I want to thank my father, mother, and uncle for their unwavering emotional support throughout this journey. A special thanks to my uncle for his guidance during my moments of confusion, which enriched my life experiences. Although the future remains uncertain, I am no longer afraid but rather emboldened to continue moving forward!

Vaasa, May 12, 2024

Ke Qiu

ABSTRACT

Author	Ke Qiu
Title	Personal Intelligent Assistant Based on Large Language Model Personalized Knowledge Extraction and Query Answering Using Local Data and Large Language Model
Year	2024
Language	English
Pages	60
Name of Supervisor	Magnus Sundell

As large language models (LLMs) become increasingly capable of understanding and generating natural language, the potential application of these models in the field of personal intelligent assistants is receiving increasing attention. This thesis proposes a new approach to building personal intelligent assistants that combines the advantages of local data sources with LLMs to provide personalized knowledge extraction and query answering services.

The system integrates RAG technology and an LLM using LangChain, utilizing local files on the user's personal computer as a knowledge source. First, RAG technology is used to extract file chunks related to user questions. These snippets are then fed into an LLM along with the original query, and the model organizes the language to generate fluent and contextual answers.

There are three main advantages of this approach: (1) It enables personalized knowledge extraction based on the user's unique data sources. (2) It leverages the powerful language understanding and generation capabilities of LLMs to provide high-quality responses. (3) It improves users' ability to quickly acquire knowledge in daily work and study.

In addition to studying how to combine large language models with local data to promote users to efficiently obtain personalized information, we have also conducted forward-looking thinking on the future expansion and improvement directions of the system. We believe that this approach will provide a new perspective on the future direction of intelligent assistants.

Keywords	AI, LLM, LangChain, RAG, information retrieval
----------	--

CONTENTS

ABSTRACT

1	INTRODUCTION	8
1.1	Background	8
1.2	Research Question	9
1.3	Scope	10
1.4	Structure of the Thesis	11
2	THEORETICAL FRAMEWORK	12
2.1	Large Language Models	12
2.1.1	Introduction to Large Language Models	12
2.1.2	How Large Language Models Work	13
2.1.3	Attention Mechanisms	14
2.1.4	Transformers	15
2.1.5	Prompt Engineering	17
2.2	Retrieval Augmented Generation (RAG)	18
2.2.1	Introduction to RAG	18
2.2.2	RAG's Workflow	19
2.2.3	Embedding	21
2.2.4	Vector Database	23
2.3	LangChain	24
2.3.1	Introduction to LangChain	24
2.3.2	Benefits of LangChain	25
2.3.3	Core Components of LangChain	26
2.4	FastAPI	29
2.5	Next.js	30
3	APPLICATION	31
3.1	Application Infrastructure	31
3.1.1	Documents Loading and Splitting	32

3.1.2	Embedding	33
3.1.3	Storage	34
3.1.4	Question Vectorization, Retrieval and Generating Answer	35
3.2	Implementation	36
3.2.1	Passing Folder Path	36
3.2.2	Documents Loading and Splitting	37
3.2.3	Vectorizing Documents and Persisting Data	38
3.2.4	Definition of Contextual Compression Retriever	39
3.2.5	Defining Prompt Template	40
3.2.6	Defining ConversationBufferMemory	40
3.2.7	Creating a ConversationalRetrievalChain Instance	41
3.2.8	Verifying Folder Selection Before Processing User Queries	42
3.2.9	Interacting with User Questions	42
3.2.10	Presentation of Answer and Sources	43
3.3	Challenges Encountered	44
3.3.1	Hardware Limitations in Local Deployment of LLM	44
3.3.2	Irrelevance Between User Queries and Document Contents	45
3.3.3	How to Increase the Credibility of LLM Answers	46
3.3.4	Vectorization Processing Challenges	46
4	RESULTS AND ANALYSIS	48
4.1	Function Implementation	48
4.2	Performance Analysis	49
4.3	The Ethics	52
4.4	Limitations	53
5	CONCLUSIONS	55
5.1	Conclusions	55
5.2	Reflections	55
5.3	Future Work	56
	REFERENCES	58

LIST OF FIGURES AND TABLES

Figure 1. The Transformer - model architecture (Vaswani, et al., 2017, p. 3).	15
Figure 2. A representative instance of the RAG process applied to question answering (Gao, et al., 2023, p. 3).	19
Figure 3. Indexing process diagram (LangChain, n.d.a).	20
Figure 4. Retrieval and generation process diagram (LangChain, n.d.a).....	21
Figure 5. The vector representation process of text (Neelakantan, et al., 2022). ..	22
Figure 6. Question retrieval process example (Cohere, 2024a).	22
Figure 7. Vector Stores Diagram (LangChain, n.d.c).	23
Figure 8. Example of creating a prompt template (LangChain, n.d.e).	26
Figure 9. Two basic operations of the memory system: reading and writing (LangChain, n.d.f).	28
Figure 10. Application infrastructure diagram.	31
Figure 11. Front-end initial page.....	37
Figure 12. Implementation of documents loading and splitting code.	38
Figure 13. Implementation of code to vectorize documents and persist data. ...	38
Figure 14. Implementation of Contextual Compression Retriever code.....	39
Figure 15. Definition of prompt template code.....	40
Figure 16. Definition of ConversationBufferMemory code.	41
Figure 17. Creating a ConversationalRetrievalChain instance code.....	41
Figure 18. Implementation of code to determine whether the user has selected a folder.	42
Figure 19. Implementation of Code for Interacting with User Queries.	43
Figure 20. Final presentation of the front-end interaction page.	43
Figure 21. Screenshot of front-end page displaying answers and their sources. ..	44
Figure 22. Example of answer when the user query is not relevant to the content of the documents in the selected folder.....	45
Figure 23. Example of answer source.	46
Figure 24. Example 1: questions requiring precise explanation.....	51
Figure 25. Example 2: questions requiring precise explanation.....	51

Figure 26. Remind users on the user interface not to upload private information.	52
---	----

Table 1. Test Results for Documents Vectorization, Retrieval and Response Generation Time.	50
---	----

1 INTRODUCTION

1.1 Background

With the continuous development of artificial intelligence technology, LLMs have become an important research direction in the field of natural language processing. In particular, OpenAI launched the conversational artificial intelligence large language model ChatGPT in November 2022, ushering in the intelligent revolution brought by LLMs. Some experts believe that this round of intelligent revolution will be comparable to the industrial revolution and the electric power revolution, profoundly changing the production and lifestyle of human society (Chen et al., 2023). After OpenAI launched ChatGPT, major international companies have also launched their own LLMs one after another, not wanting to fall behind in this round of revolution.

These LLMs have amazing capabilities. They can not only conduct conversations fluently and answer complex questions, but also generate high-quality text, pictures, and videos. They can help people write promotional copy, write code, create videos, and more. Different from traditional regular intelligent assistants, LLMs give intelligent assistants a more vivid "soul", making their responses no longer rigid but full of emotion, insight and imagination. At the same time, the extensive knowledge and powerful reasoning capabilities of LLMs help make intelligent assistants more insightful and efficient in solving various complex problems in the real world.

However, LLMs still face some problems. An LLM requires a large amount of data and computing power for training, and the cost is extremely high. Taking ChatGPT 3 as an example, the cost of training once is approximately \$1.4 million (Liu & Yang, 2023, p.5). Because the cost is too high, the knowledge update of LLMs will be relatively slow. In other words, an LLMs does not know the latest news. In addition, ChatGPT provided by OpenAI suffers from "hallucination" phenomenon. When ChatGPT is faced with a question it does not understand, it will make up an answer.

On the surface it appears to be a reasonable answer, but actually contains incorrect information. This requires personal judgment, and its answer cannot be fully trusted. (Bowman, 2023)

For the problem of large language model hallucinations, there are currently five main correction methods, namely Parameter Adaptation, Post-hoc Attribution and Edit Technology, Leverage External Knowledge, Assessment Feedback, and Mindset Society (Ye et al., 2023, pp. 7-10). They respectively alleviate the hallucination problem from the perspectives of adjusting model parameters, guiding inference, retrieving external knowledge, strengthening feedback evaluation, and collaborative debate between models. The problem of slow knowledge updating for LLMs can be solved by fine-tuning pre-trained models and using Retrieval Augmented Generation (RAG) technology. They can be used separately or complement each other. In this thesis, RAG was used technology to combine a large language model with a personal knowledge base to build an intelligent assistant.

1.2 Research Question

The research question across from a very common problem of locating files on the computer. During the course of work, a significant number of files accumulate on the computer over time. A common issue encountered is the difficulty in locating specific files when needed, often resulting in considerable time spent searching for them. This challenge is likely shared by many workers in today's highly digitalized society. Thus, the focus of this thesis is:

- How to effectively integrate Retrieval-Augmented Generation and large language model to improve the efficiency and accuracy of file retrieval in personal computer file systems and answer user queries about file content in real time?

To solve this problem, research papers in related fields were consulted. A paper from Sharma and Kumar (2023) has similar research content. The study retrieved

relevant documents through keywords entered by users, mainly using ontology to define the semantic relationship between words, and used Word2vec to convert the words in the document into vectors, and then filtered relevant documents through similarity calculation. The study stated that the execution time of this method was within 1.7 seconds, which improved the similarity index and accuracy of the query results.

The approach used in this thesis integrates Retrieval Augmented Generation (RAG) with LLM. This approach distinctively segments documents into semantically coherent chunks, which are then vectorized and stored in a vector database. During the retrieval phase, user queries are also transformed into vector form and matched against this database to locate relevant document chunks. This technique enhances the relevance of the document chunks retrieved in response to the user queries. Additionally, these retrieved chunks are utilized as contextual data for the LLM to generate personalized responses. However, the use of an LLM to generate answers introduces greater complexity and, consequently, increases computational demands, which may extend the execution time.

This application can help users quickly find relevant files, and can also answer users' questions about the file. At the same time, the combination of local files and LLM also makes up for the problem of slow knowledge update of LLMs. If such an application is deployed in an enterprise, it can also improve employee work efficiency and save costs for the enterprise (Paschen et al., 2020).

1.3 Scope

The research focus of this thesis is to effectively integrate RAG with LLM to improve the efficiency and accuracy of file retrieval in personal computer file systems. The research contents include reviewing other people's previous research, document segmentation and vectorization technology, SimilaritySearch, the use of LLMs, and optimization of prompt. Among them, the documents on the local computer mainly include PDF, Word, and TXT files. Through intelligent processing of

personal document resources and knowledge base construction, it can help users quickly retrieve files. At the same time, LLM can be equipped with domain-specific expertise to better serve individual needs.

1.4 Structure of the Thesis

This thesis has five chapters in total. The first chapter is the introduction, explaining the research background and research scope. Chapter 2 provides an overview of related technologies, such as LLMs, LangChain, and RAG. Chapter 3 introduces the application architecture, key technologies and implementation details in detail. Chapter 4 gives the experimental results and analysis of areas that need improvement. Chapter 5 is the summary and outlook for future work.

2 THEORETICAL FRAMEWORK

This chapter mainly introduces the large language model and related basic concepts of artificial intelligence, as well as the frameworks used in the back-end and front-end respectively.

2.1 Large Language Models

2.1.1 Introduction to Large Language Models

A large language model (LLM) is a language model notable for its ability to achieve general-purpose language generation and other natural language processing tasks such as classification. LLMs acquire these abilities by learning statistical relationships from text documents during a computationally intensive self-supervised and semi-supervised training process. LLMs can be used for text generation, a form of generative AI, by taking an input text and repeatedly predicting the next token or word. In addition, LLMs can also generate images and videos. (Wikipedia, 2024.)

LLMs are artificial neural networks. The largest and most capable, as of March 2024, are built with a decoder-only transformer-based architecture while some recent implementations are based on other architectures, such as recurrent neural network variants and Mamba (a state space model). (Wikipedia, 2024.)

Up to 2020, fine tuning was the only way a model could be adapted to be able to accomplish specific tasks. Larger sized models, such as GPT-3, however, can be prompt-engineered to achieve similar results. They are thought to acquire knowledge about syntax, semantics and "ontology" inherent in human language corpora, but also inaccuracies and biases present in the corpora. (Wikipedia, 2024.)

Some notable LLMs are OpenAI's GPT series of models (for example, GPT-3.5 and GPT-4, used in ChatGPT and Microsoft Copilot), Google's PaLM and Gemini (the latter of which is currently used in the chatbot of the same name), xAI's Grok,

Meta's LLaMA family of open-source models, Anthropic's Claude models, Mistral AI's open-source models, and Databricks' open source DBRX. (Wikipedia, 2024.)

2.1.2 How Large Language Models Work

LLMs are neural network-based machine learning models designed to learn from large amounts of text data to predict the next word in any given context. This task of predicting the next word is called language modeling. Neural networks can learn arbitrarily complex mapping relationships, so they can well solve this seemingly simple but actually highly complex language modeling problem.

The training of LLMs is divided into three stages. The first stage is pre-training. At this stage, LLMs will conduct unsupervised training on massive amounts of original text data (such as Internet data, books, or research thesis) and learn to predict the next word in various contexts. This process does not require manual annotation of data. It only requires dividing the original data into multiple sequence samples and inputting them into the model. Pre-training enables LLMs to acquire language understanding capabilities such as grammar, semantic understanding, and extensive world knowledge. (Stöffelbauer, 2023.)

The second stage is instruction fine-tuning. In order for pre-trained LLMs to be able to answer questions and execute instructions as an intelligent assistant, further instruction fine-tuning training is required on high-quality question-answer data pairs. The acquisition cost of this instruction data set is high because high-quality data pairs need to be manually constructed. After fine-tuning the instructions, LLMs are no longer a simple text generator, but can understand and respond to various instructions and queries. (Stöffelbauer, 2023.)

Some top LLMs will also undergo a third stage of human feedback reinforcement learning after instruction fine-tuning. At this stage, humans will evaluate the quality of LLMs' output, and this feedback information is used to continue training LLMs to make their output more consistent with human values and preferences.

Combined with instruction fine-tuning and human feedback reinforcement learning, the output quality and human understandability of LLMs will be further improved. (Stöffelbauer, 2023.)

2.1.3 Attention Mechanisms

Attention mechanisms play a crucial role in the field of artificial intelligence, especially when processing sequence data such as natural language. This mechanism enables the model to selectively focus on different parts of the input when processing the data, by generating attention scores that represent the degree of attention the model pays to each part of the data. This process involves generating a weighted sum of input data, which in turn affects the next step of processing of the model, enabling the model to not only capture key information in the data, but also understand the context and relationships within the data, addressing the limitations of traditional sequence processing methods when facing complex data. (XQ, 2024a.)

The concept of attention mechanisms originated from improvements on early neural network translation models, which attempted to encode input sentences into fixed-length vectors and then decode these vectors to generate translations. As the length of the input sentence increases, the performance of this approach starts to degrade because fixed-length vectors struggle to capture all the input details. To overcome this challenge, the researchers proposed methods that utilize a sequence of vectors instead of a single fixed-length vector, allowing the model to focus on the most critical information when processing long text. (Bahdanau et al., 2014.)

Among them, scaling dot product attention is a method to implement the attention mechanism. It calculates the dot product of the query, key and value, and then scales the dot product result and applies the SoftMax function to determine the attention weight of each part, so that the model is able to focus on the most

The Transformers architecture consists of two parts: encoder and decoder (As shown in Figure 1, the encoder is on the left and the decoder is on the right), each part is stacked by multiple identical layers. Each layer contains Multi-Head Attention and Feed-Forward Neural Network, and effective training of deep networks is achieved through Residual Connection and Layer Normalization.

- The encoder is responsible for processing the input sequence, capturing the relationships within the sequence through the self-attention mechanism, and converting the input into representations in high-dimensional space. These representations contain the deep semantic information and contextual relationships of the input. (XQ, 2024b.)
- The decoder is based on the output of the encoder and the previously generated output, the final output is generated through self-attention and Encoder-Decoder Attention mechanisms. The decoder takes into account the contextual information of the entire input sequence when generating each output element, enabling high-quality sequence generation. (XQ, 2024b.)

Training of Transformers models involves adjusting millions or even billions of parameters to minimize the difference between predicted and actual outputs. The trained model can demonstrate human-level performance on a variety of tasks, including but not limited to machine translation, text generation, language understanding, and question answering systems. (XQ, 2024b.)

Through its innovative self-attention mechanism, the transformers architecture not only solves the problem of long-distance dependencies, but also greatly improves the parallel processing capabilities of the model, ushering in a new era of natural language processing and other sequence processing tasks. The proposal and successful application of this architecture marks an important milestone in the field of deep learning and artificial intelligence, laying a solid foundation for future research and applications.

2.1.5 Prompt Engineering

Prompt engineering is the practical discipline of developing and optimizing prompts to effectively utilize LLMs to perform natural language processing tasks. This discipline involves the creation of task-specific prompts that guide LLMs to produce desired results by embedding the task into the input itself. Prompt engineering is critical to the ability to leverage LLMs in a variety of applications, from text generation to image synthesis, improving their performance. (Giray, 2023.)

Prompt words serve as instructions or queries that direct the behavior of LLMs to produce specific outputs. They are critical in guiding the focus of LLMs, enabling them to generate contextually relevant and accurate responses. By using carefully crafted prompt words, users can improve the quality of output in tasks such as content creation, software development, and more, making interactions with LLMs more effective and tailored to specific needs. To demonstrate the power of prompt engineering, we reference the following example from White et al. (2023):

Prompt: "From now on, I would like you to ask me questions to deploy a Python application to AWS. When you have enough information to deploy the application, create a Python script to automate the deployment."

According to White et al. (2023), this example prompt causes ChatGPT to begin asking the user questions about their software application. ChatGPT will drive the question-asking process until it reaches a point where it has sufficient information to generate a Python script that automates deployment. This example demonstrates the programming potential of prompts beyond conventional "generate a method that does X" style prompts or "answer this quiz question".

To use prompts effectively, the elements of a prompt must be considered, including instructions, context, input data, and output instructions. Giray (2023) explains these terms as:

(1) Instruction is a specific task or instruction that guides the model's behavior and directs it toward the desired output.

(2) Context External refers to information or additional context that provides background knowledge to the model, helping it generate more accurate and relevant responses.

(3) Input data means the input or question that we want the model to process and provide a response for. It forms the core of the prompt and drives the model's understanding of the task.

(4) Output indicator specifies the type or format of the desired output. It helps shape the response by defining whether we need a short answer, a paragraph, or any other specific format.

These elements help accurately convey the user's intent to the language model, prompting the generation of targeted responses. Users can improve their prompt engineering skills by understanding and implementing various prompt patterns, such as instructive, contextual, and mixed prompts, to improve interactions with LLMs and achieve desired results.

2.2 Retrieval Augmented Generation (RAG)

2.2.1 Introduction to RAG

Retrieval Augmented Generation or RAG, refers to a technology that provides additional information to LLM through external knowledge sources. Based on the user's query, RAG retrieves relevant document chunks from external professional knowledge bases and then provides them to an LLM. LLM uses the retrieved document chunks as context for answering user questions. This empowers the LLM to answer questions it does not know, reducing the likelihood of generating inaccurate or fictitious content.

We can better understand RAG with an example. For an LLM, RAG is equivalent to a human open-book exam. In open-book exams, students can bring reference materials such as textbooks and notes, and even check online. When they encounter a question they do not know, they can look up the information and use reasoning to finally get a more accurate answer.

2.2.2 RAG's Workflow

The RAG workflow is divided into three stages: indexing, retrieval, and generation. The output of each stage has an important impact on the next stage. Specifically, this process begins by indexing the information into a searchable vector format. Then, relevant data is retrieved based on the user's query. Finally, based on the retrieved data, LLM is responsible for generating answers. These three stages are explained in more detail in the following sections.

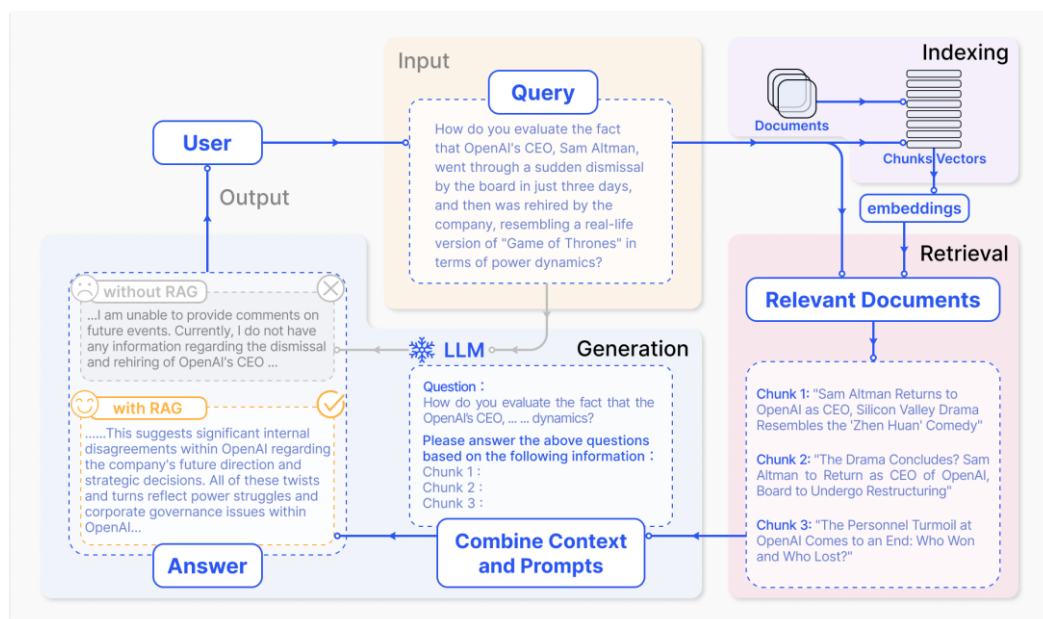


Figure 2. A representative instance of the RAG process applied to question answering (Gao, et al., 2023, p. 3).

The initial stage of the RAG workflow is the indexing stage. During this process, various files (which may include documents, web pages, videos, and other forms of content) are split into manageable chunks. Each chunk is then converted into a

vector and stored in a vector database for efficient subsequent data retrieval. Converting various data into vectors at this stage is critical as it underpins the performance and accuracy of the subsequent retrieval stage. (Gao, et al., 2023, p. 3.)

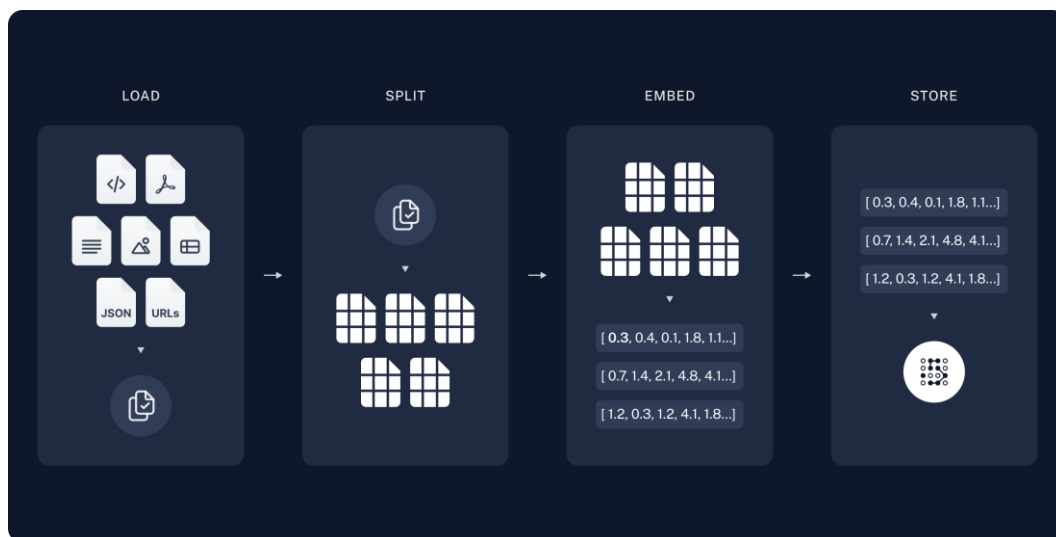


Figure 3. Indexing process diagram (LangChain, n.d.a).

After indexing, the RAG workflow enters the retrieval phase. This critical step involves accessing the vector database to find chunks relevant to the user query. First, the user's query is converted into a vector, and then the converted vector is used to retrieve the top "k" chunks in the vector database, where "k" represents a predefined number of chunks. The chunks are matched based on the similarity between the query and the chunk. The retrieval stage is crucial because the quality and relevance of the selected chunks directly affect the quality of the subsequent LLM-generated answers. (Gao, et al., 2023, p. 3.)

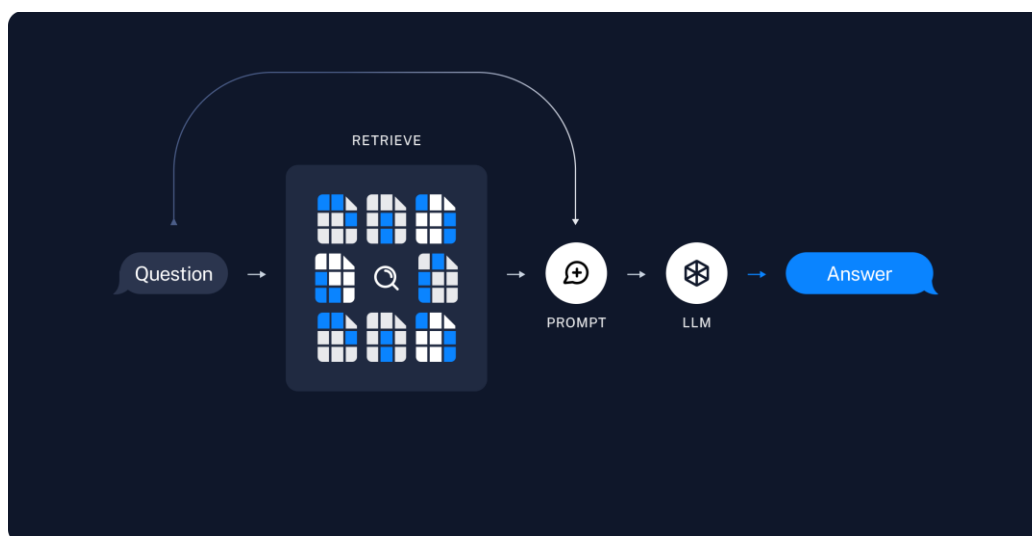


Figure 4. Retrieval and generation process diagram (LangChain, n.d.a).

The final step in the RAG workflow is the generation phase. At this stage, the LLM is used to synthesize the information obtained during the retrieval stage. Specifically, the selected chunks and the initial query are combined into prompts and then input into the LLM. The LLM generates linguistically coherent answers for direct use by end users by leveraging previously retrieved information. (Gao, et al., 2023, p. 3.)

The above is the workflow of RAG. By introducing external data sources, the possibility of inaccurate LLM answers is reduced. As a result, the answers generated by LLM have higher credibility and better meet the user's requirements.

2.2.3 Embedding

Embedding is an important technology in RAG. It is a technology that converts text or other format data into vector representation. These vectors are usually represented in a multidimensional space, where similar words or phrases are close together in the vector space. In this way, embeddings are able to capture and express the semantic and grammatical features of words, enabling computers to understand and process natural language.

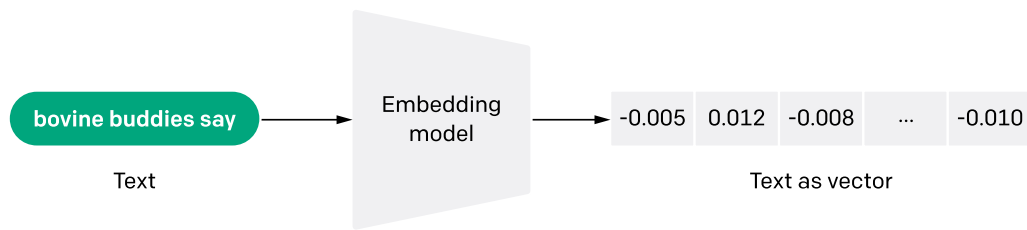


Figure 5. The vector representation process of text (Neelakantan, et al., 2022).

In RAG, both the user's question and the document chunks are converted into vector representations. Retrieval is achieved by calculating the similarity (for example, cosine similarity) between the embedding vectors of the question and the document chunks.

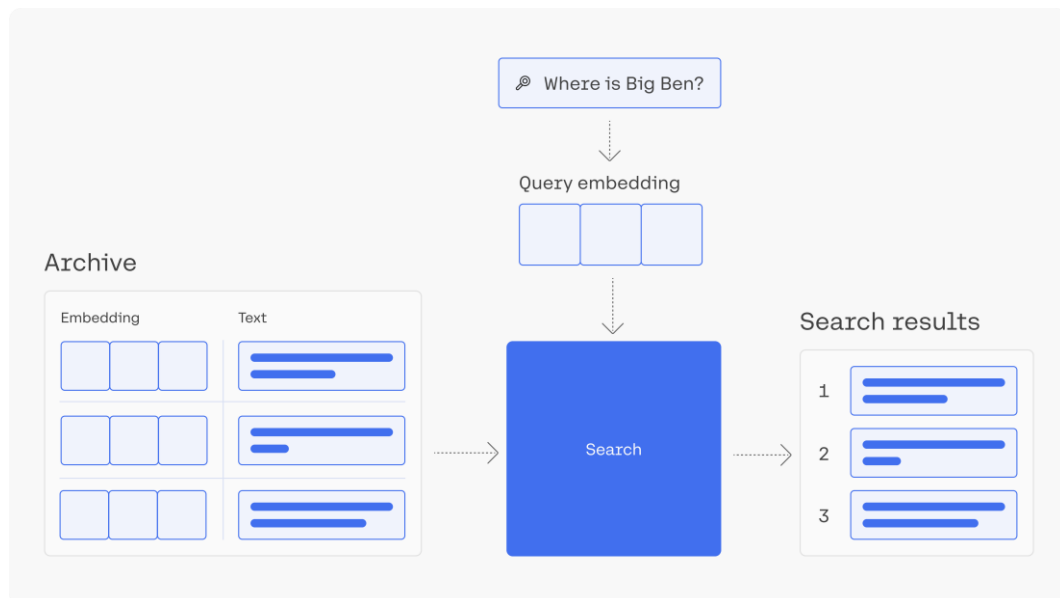


Figure 6. Question retrieval process example (Cohere, 2024a).

For example, suppose there is a question "Where is Big Ben?" The system first converts this question into a vectorized format and then compares the similarity with the vectorized data of all documents in the vector database. In this process, the semantic representation ability of the embedding model plays a key role (Gao, et al., 2023, p.9). The embedding model is a model that specifically implements mapping any type of raw data (such as text, images, and sounds) to a continuous vector space of fixed length.

2.2.4 Vector Database

Vector database is also an important technology involved in RAG. A vector database is a database specifically designed to store and retrieve high-dimensional vector data. These databases are capable of efficiently managing complex unstructured data such as text, images, and audio. The vectors generated by the embedding model are stored in the vector database, which avoids the need to vectorize the knowledge base every time the embedding model is used. In addition, vector databases provide efficient indexing and retrieval mechanisms for these vectors, and are particularly good at handling similarity searches, such as finding the set of vectors that are closest to a given vector.

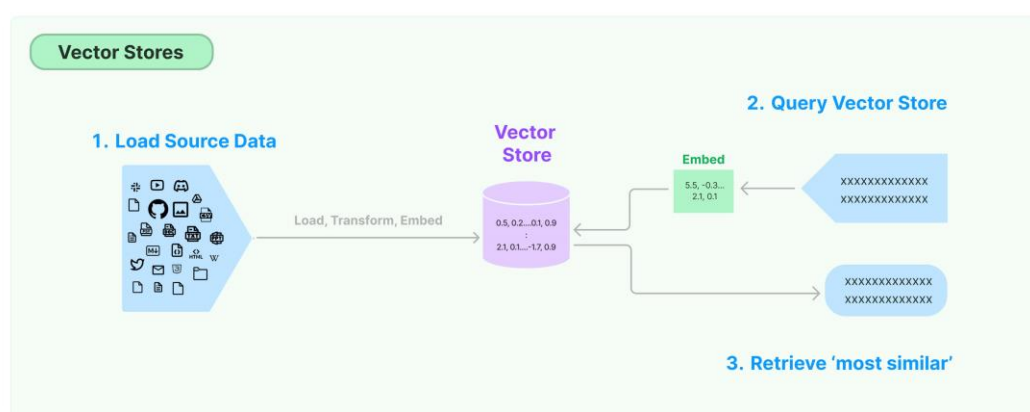


Figure 7. Vector Stores Diagram (LangChain, n.d.c).

Search and retrieval technology are the core functions of vector databases. Approximate Nearest Neighbor Search (ANNS), which are fundamental operations in vector databases, aiming to find the nearest data points in high-dimensional spaces. This is crucial for tasks such as recommendation systems and image retrieval. There are also hash-based, tree-based and graph-based methods. They are different methods for organizing and indexing data to optimize search operations (Han, et al., 2023). For applications with high real-time requirements, graph or hash methods may be preferred. In scenarios where higher accuracy is required, tree-based methods may be chosen.

Although the vector database has excellent search and retrieval technology, it can handle complex multi-dimensional data and provide fast retrieval capabilities, which is especially important for data-driven applications. However, vector databases also face some challenges, such as how to handle increasingly larger data sets while maintaining high performance, and how to ensure the accuracy and efficiency of searching in high-dimensional spaces. (Han, et al., 2023, p. 9.)

In this thesis, vector database plays a key role. We will use vector database to store vector representation of document chunks. In addition, vector database will also be used to find document chunks related to user questions with its efficient indexing and retrieval mechanism. This technology can improve the accuracy and efficiency of document retrieval and provide strong support for the smooth development of research.

2.3 LangChain

2.3.1 Introduction to LangChain

LangChain is a framework for developing applications powered by LLMs (LangChain, n.d.d). It simplifies all stages of the LLM application lifecycle. In the development phase, LangChain provides open-source building blocks and components that enable developers to quickly build applications. At the same time, it also integrates third-party tools and templates to further improve development efficiency. In the productization phase, LangChain provides the LangSmith tool for inspecting, monitoring, and evaluating chain applications, which helps with continuous optimization and high-confidence deployment. Finally, with the LangServe tool, any chain application can be quickly converted into an API service for deployment. The framework provides full lifecycle support for the development, optimization, and deployment of LLM applications. (LangChain, n.d.d.)

LangChain works by simplifying complex LLM application programming into easily manageable modular components. LangChain encapsulates one or more complex

processes as components. These components can be "chained" together to create applications that minimize the amount of code and deep understanding required to perform complex NLP tasks. While LangChain's abstract approach may limit the degree to which expert programmers can finely customize applications, it enables experts and novices alike to quickly experiment and prototype.

2.3.2 Benefits of LangChain

With LangChain, organizations can repurpose LLM for domain-specific applications without the need for retraining or fine-tuning. Development teams can build complex applications that reference proprietary information to enhance model response. For example, developers can use LangChain to build applications that read data from stored internal documents and aggregate it into conversational responses. RAG workflows can also be created to introduce new information to the language model when prompted. Implementing context-aware workflows such as RAG can reduce model hallucinations and improve response accuracy. (Amazon Web Services, n.d.)

LangChain simplifies the development of AI by abstracting the complexities of data source integration and prompt refinement. Developers can customize sequences to quickly build complex applications. Software teams can modify the templates and libraries provided by LangChain without programming business logic, thereby reducing development time. (Amazon Web Services, n.d.)

Additionally, LangChain provides AI developers with tools to connect language models with external data sources. LangChain is open source and powered by an active community. Organizations can use LangChain for free and receive support from other developers who are proficient in the framework. (Amazon Web Services, n.d.)

2.3.3 Core Components of LangChain

Firstly, the first core component of LangChain is introduced: Prompts. A prompt for a language model is a set of instructions or input provided by a user to guide the model's response, helping it understand the context and generate relevant and coherent language-based output, such as answering questions, completing sentences, or engaging in a conversation.

LangChain provides tooling to create and work with prompt templates. A template may include instructions, few-shot examples, and specific context and questions appropriate for a given task. The following is an example of using a template provided by LangChain to create prompt words:

```
from langchain_core.prompts import PromptTemplate

prompt_template = PromptTemplate.from_template(
    "Tell me a {adjective} joke about {content}."
)
prompt_template.format(adjective="funny", content="chickens")
```

Figure 8. Example of creating a prompt template (LangChain, n.d.e).

Retrieval is another core component of LangChain. Through the RAG method, LangChain retrieves external data and passes it to LLM to satisfy the LLM application's need for specific data. LangChain provides all the building blocks for RAG applications - from simple to complex. It contains several key modules: Document loaders, Text Splitting, Text embedding models, Vector stores, Retrievers, Indexing. LangChain (n.d.b) introduces these modules respectively.

Document loaders are used to load documents from many different sources. LangChain offers more than 100 different document loaders capable of handling various types of documents (HTML, PDF, code) from different locations (private S3 buckets, public websites).

Text Splitting involves splitting long documents into smaller chunks that fit within the model's context window. This process ensures that the text is manageable and can be effectively processed by the LLM. By breaking down larger texts, it helps maintain the relevance and context within the model's capacity.

Text embedding models convert documents into vector representations, where vectors capture the semantics of text, making it easy to quickly and efficiently find similar chunks of text in vector space.

Vector stores are used to store vector representations of text and perform vector searches. These stores efficiently handle large volumes of vector data, enabling rapid retrieval of information. By leveraging vector stores, LangChain can provide quick access to relevant data based on semantic similarity.

Retrievers are crucial, as they retrieve text chunks related to a given query from a vector database and are considered the core components of LangChain. They ensure that the most relevant information is accessed promptly in response to user queries. This functionality is essential for maintaining the efficiency and accuracy of data retrieval in LLM applications.

The Indexing module can be used to synchronize documents into a vector database. The primary purpose of Indexing is to prevent duplicate content from being written to the vector store, avoid rewriting unchanged content, and prevent embeddings from being recomputed on unchanged content.

The third core component of LangChain is Chains. Chains are at the heart of LangChain's workflow. They combine LLM with other components to create applications by performing a series of functions.

The most basic chain is LLMChain. It just calls a model and the hint template for that model. Taking OpenAI's LLM application as an example, assume there is a prompt template named "ExamplePrompt". Users can import LLMChain from the "langchain.chains" module and then create an instance:

```
chain_example = LLMChain (llm = OpenAI (), prompt = ExamplePrompt)
```

When the chain needs to be run to handle a specific “input”, the user can simply execute:

```
chain_example.run (“input”)
```

Here is a simple demonstration of the basic application of the chain. The LLMChain can be easily integrated into more complex workflows by connecting it with other chains or components, enabling the development of sophisticated applications. This flexibility and modularity make LangChain a powerful tool for leveraging LLM capabilities.

Finally, the fourth core component of LangChain is Memory. Most LLM applications have a conversational interface. An essential component of a conversation is being able to refer to information introduced earlier in the conversation. At bare minimum, a conversational system should be able to access some window of past messages directly. We call this ability to store information about past interactions “memory”.

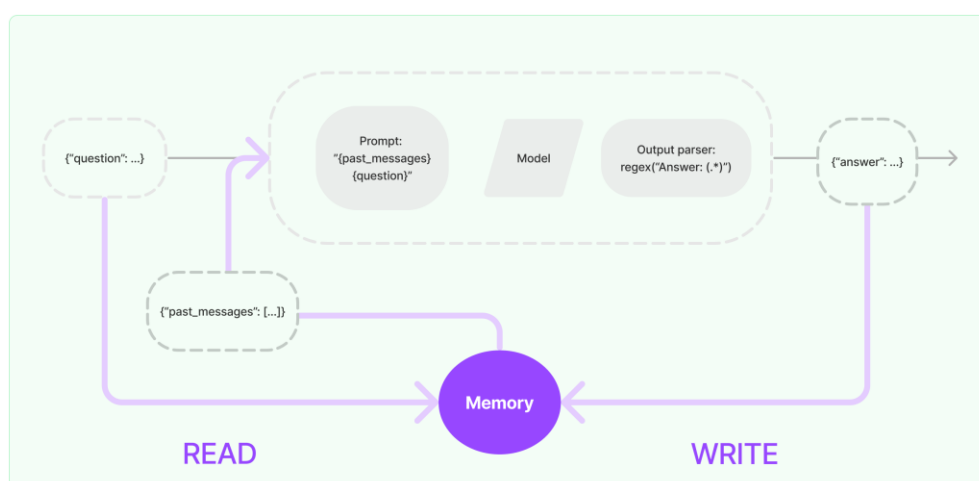


Figure 9. Two basic operations of the memory system: reading and writing (LangChain, n.d.f).

A memory system needs to support two basic actions: reading and writing. Recall that every chain defines some core execution logic that expects certain inputs. Some of these inputs come directly from the user, but some of these inputs can come from memory. A chain will interact with its memory system twice in a given run.

- AFTER receiving the initial user inputs but BEFORE executing the core logic, a chain will READ from its memory system and augment the user inputs. (LangChain, n.d.f.)
- AFTER executing the core logic but BEFORE returning the answer, a chain will WRITE the inputs and outputs of the current run to memory, so that they can be referred to in future runs. (LangChain, n.d.f.)

2.4 FastAPI

FastAPI is a modern, fast (high-performance), web framework for building APIs with Python 3.8+ based on standard Python type hints (FastAPI, n.d.). Considered in web frameworks, it is the third popular (based on the Github stars and forks) after Django and Flask. It is quite new compared to other giants such as Django and Flask that the project started in 2019. However, it gained a huge popularity among community.

The framework offers exceptionally high performance, comparable to NodeJS and Go, owing to its integration with Starlette and Pydantic, making it one of the fastest available Python frameworks (FastAPI, n.d.). This performance boost allows developers to handle more requests per second compared to many other frameworks. Consequently, applications built with FastAPI can scale efficiently to meet high demand.

FastAPI significantly accelerates the development process, enhancing feature development speed by approximately 200% to 300% (FastAPI, n.d.). This is achieved

through its intuitive design and automatic interactive API documentation. As a result, developers can quickly implement and test new features, reducing time-to-market.

The framework is designed to decrease developer-induced errors by roughly 40%, contributing to more reliable code (FastAPI, n.d.). This reduction in errors is primarily due to FastAPI's strong typing system and validation mechanisms. By catching potential issues early, developers can produce cleaner and more maintainable code.

It provides excellent support for code editors, with comprehensive autocompletion features, reducing debugging time (FastAPI, n.d.). These features help developers write code faster and with fewer mistakes. Enhanced editor support also means that even less experienced developers can contribute effectively to a project.

The framework is user-friendly, simplifying the learning process and reducing the time needed to consult documentation (FastAPI, n.d.). It's clear and concise documentation is easy to navigate, making it accessible to both beginners and experienced developers. This ease of use encourages more developers to adopt FastAPI for their projects.

2.5 Next.js

Next.js is a React framework for building full-stack web applications. It leverages React components for crafting user interfaces while offering a suite of additional features and optimizations to enhance performance and scalability.

One of the main advantages of Next.js is that it supports server-side rendering. This means that the server can generate the HTML for the page and send it to the client, rather than using JavaScript to generate the HTML on the client side. This ensures that even dynamically generated content loads and renders quickly, improving user experience.

3 APPLICATION

This chapter introduces the architecture of the personal intelligent assistant application in detail and provides an in-depth analysis of its various components. In addition, this chapter also explains the implementation details of each stage of application development, as well as the challenges encountered during the development process and their solutions.

3.1 Application Infrastructure

This project develops a personal intelligent assistant application by integrating RAG, LangChain and LLM. This application enables users to interact directly with files in folders on their personal computers in a conversational manner, thereby achieving the goal of quickly retrieving and querying file contents. Through this innovative interaction method, users can manage and query computer files more efficiently, significantly improving work efficiency and file processing capabilities.

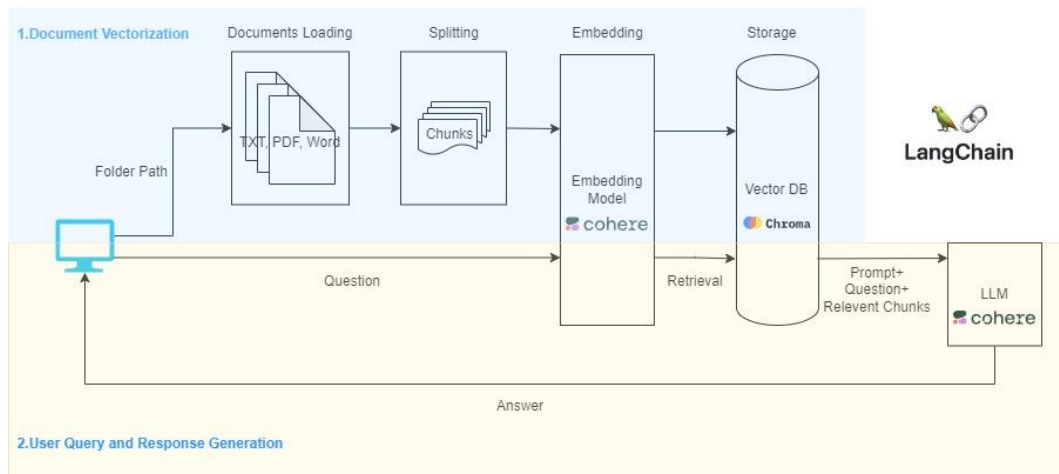


Figure 10. Application infrastructure diagram.

The application is developed based on the LangChain framework and mainly includes two core parts: Document Vectorization, User Query and Response Generation. In the document vectorization process, the front-end first transmits the folder path selected by the user to the back-end. Then, the back-end loads the files

based on the path, and splits and vectorizes the files. Finally, the processed data is stored in the vector database.

In the user query and response generation system, the front-end first transmits the user's query question to the back-end. The back-end then vectorizes the question and retrieves the top “k” chunks that are most similar to the question vector from the vector database. These questions and the retrieved data chunks are combined into context and added to the prompt together, and then submitted to the LLM to generate the answer. Each process is described in detail below.

3.1.1 Documents Loading and Splitting

This process is to load all the files in the folder according to the folder selected by the user. In fact, there are many document loaders in LangChain, for example, CSVLoader, PyPDFLoader, TextLoader, which load CSV documents, PDF documents, and Text documents respectively. DirectoryLoader is selected in this thesis, because it can load all documents in the folder. The user's folder may contain documents of various formats. If we use DirectoryLoader to load documents, we do not need to distinguish the document format, which is very convenient to use. In this thesis, three types of documents are mainly involved: TXT, PDF, and Word.

After loading the documents, we need to do some processing on it so that it is better suited for the application. Because the context window of LLM is limited, we need to split the long document into smaller chunks. Splitting text into chunks is actually quite complicated. We need to consider keeping semantically related text chunks together. At a high level, text splitters work as following:

- Split the text up into small, semantically meaningful chunks (often sentences).
- Start combining these small chunks into a larger chunk until you reach a certain size (as measured by some function).

- Once you reach that size, make that chunk its own piece of text and then start creating a new chunk of text with some overlap (to keep context between chunks).

3.1.2 Embedding

After splitting a long document into chunks, the chunks need to be converted into vector representations. This is very useful for computers. It has the following two main functions.

First, it improves retrieval efficiency. The distance between vectors in space can indicate the degree of similarity between them. The closer they are to the vector, the more similar they are. Therefore, by calculating the distance between vectors, similar texts can be quickly retrieved, thereby achieving efficient information retrieval.

Second, it helps in understanding text semantics. The difference between two vectors in space is very similar to the difference between the other two vectors, which means that their relationship is very similar. For example, the vector difference between cats and mice is very similar to the vector difference between policemen and thieves, which means that the relationship between cats and mice is very similar to the relationship between policemen and thieves. Therefore, converting text into vectors can better understand the intent of the query and the content of the retrieved document, so as to find semantically matching information to support the generation process.

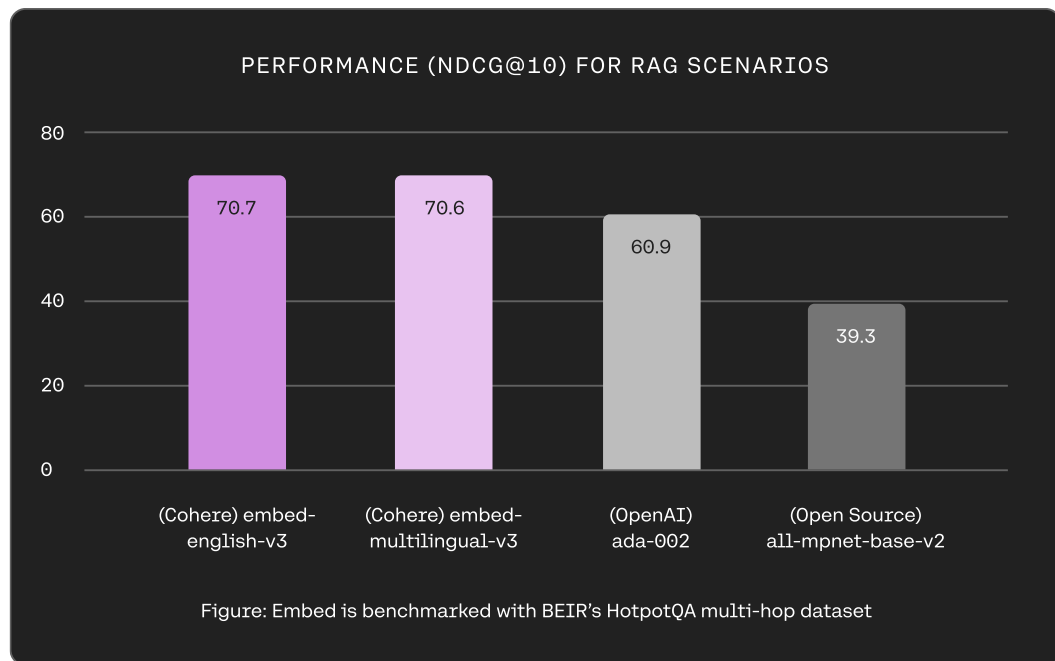


Figure 6. Cohere embedding model performance for RAG scenarios (Cohere, 2024b).

In this thesis, the Cohere embedding model was adopted to convert text into vectors. The main reason for choosing Cohere embedding model is its optimization effect on RAG retrieval. The performance improvement of Cohere embedding model helps to improve the relevance of RAG retrieval and reduce search results, thereby saving retrieval time and computing resources (Cohere, 2024b). In addition, Cohere offers free rate-limited learning usage with no fees before going into production.

3.1.3 Storage

After vectorizing the text, the vectors need to be stored in a database. Vector database is a database optimized for storing and retrieving high-dimensional vector data. In the RAG framework, its role is crucial. The main advantage of a vector database is its ability to efficiently process and store large amounts of vectorized data. They usually use special data structures and indexing strategies to effectively organize and retrieve vector data. Vector databases can handle high-dimensional data while providing approximate nearest neighbor (ANN) queries. This kind of

query can quickly find data items similar to the query vector. This enables the RAG system to quickly retrieve the information most relevant to user queries from massive data, significantly improving the speed of information processing. In addition, vector databases also play a key role in improving the accuracy of data processing. It ensures the accuracy and relevance of search results, thereby enhancing the output quality of the RAG system's generated models.

In this thesis, the Chroma vector database was utilized. The primary reasons for choosing Chroma are as follows: firstly, Chroma supports the association of rich text metadata and allows for the filtering of vector data based on these metadata conditions, which is advantageous in semantic search and question-answering systems within NLP applications. Additionally, Chroma is suitable for small-scale data systems and is user-friendly, enabling developers to get started quickly. Besides the Chroma vector database, the FAISS vector database is also an excellent choice. FAISS, an open-source library developed by Facebook, has been used internally at Facebook for many years, demonstrating stable performance, and is particularly well-suited for systems that need to handle large-scale vector data.

3.1.4 Question Vectorization, Retrieval and Generating Answer

Vectorizing the user's question facilitates subsequent retrieval of relevant file chunks from the vector database. This is a little different from the previous Embedding step. The previous Embedding step embeds documents and uses multiple texts as input. Question vectorization embeds queries and uses a single text as input.

The retrieval step is to retrieve relevant file chunks from the vector database based on the user's question. There are many retrieval algorithms. The maximum marginal relevance search (MMR) is used in this thesis. By default, the vector storage retriever uses similarity search. Using similarity search, duplicate or extremely similar content may be returned. Therefore, in this thesis, in order to ensure that

the retrieved information is relevant to the query while reducing the redundancy between results, the maximum marginal relevance search is used.

The retrieved file chunks are combined into prompts together with user questions and submitted to the LLM to generate an answer. The prompt setting is very important. For example, you can tell LLM in the prompt that if it doesn't know the answer to a question, it should not try to make up an answer, but should answer truthfully that it doesn't know.

3.2 Implementation

In this section, I will describe in detail the specific implementation details of this application. Among these, I will introduce in detail the steps of documents loading, splitting, vectorization, retrieval, and finally generating answers.

3.2.1 Passing Folder Path

In the application design of this thesis, the user first selects a specified folder on the front-end interface. Then, the system transmits the path of the selected folder to the back-end for processing. This design chooses to transmit the folder path instead of the file, mainly based on security considerations. Specifically, this application is deployed on a local server instead of a cloud server to avoid uploading files to the cloud. This is mainly because users may have reservations about uploading sensitive files to the cloud. By deploying the application locally, users' concerns about data security can be reduced, thereby enhancing their trust in the system.

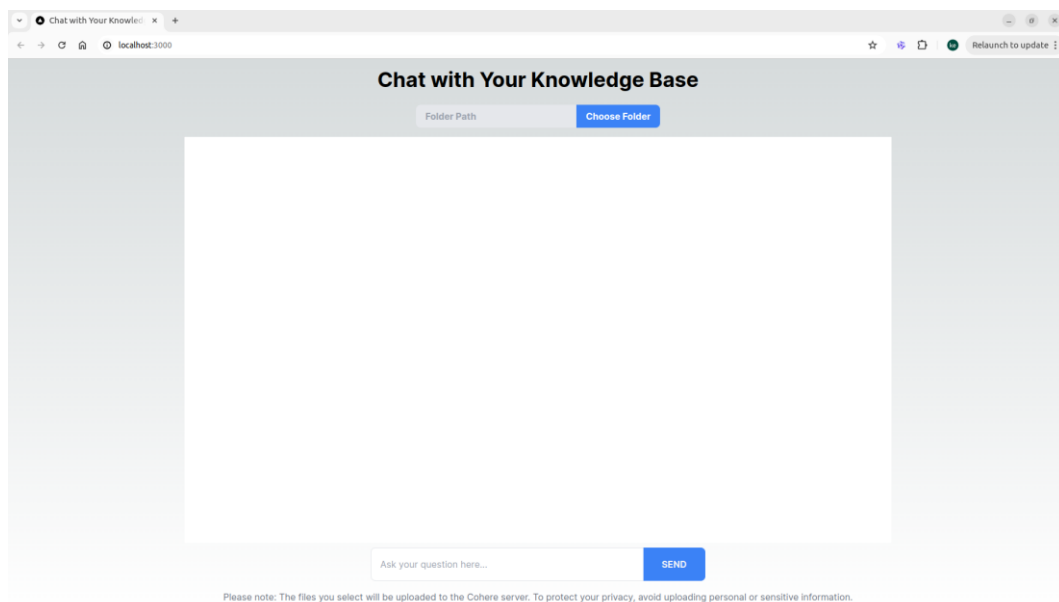


Figure 11. Front-end initial page.

3.2.2 Documents Loading and Splitting

The documents loading and segmentation process of this thesis application is shown in the figure below. First, the documents loading is implemented by DirectoryLoader, which is responsible for loading all files from the file path passed by the front end. In the implementation of DirectoryLoader, UnstructuredLoader is used by default to load each file in the folder. UnstructuredLoader supports the loading of multiple file types, including text files, PowerPoints, html, pdfs, and images.

The documents segmentation is completed by NLTKTextSplitter, which uses the tokenizer provided by NLTK (Natural Language Toolkit) to segment the text. The size of the split text block is calculated in the number of characters. NLTK is a suite of libraries and programs for symbolic and statistical NLP for English written in the Python programming language (LangChain, n.d.g). In the application of this study, the document was divided into multiple blocks of no more than 1000 characters in length. If a sentence longer than 1000 characters is encountered, the sentence will be assigned to another block without being truncated. This is a major advantage of NLTKTextSplitter. It can segment text in a semantically reasonable way,

rather than simply segmenting based on the number of characters or tokens. This makes the resulting text blocks easier to understand and process.

```
# Documents loading and splitting
def load_documents(docs_path: str):
    if not os.path.exists(docs_path):
        raise HTTPException(status_code=404, detail="Document path not found")

    loader = DirectoryLoader(docs_path)
    docs = loader.load()
    text_splitter = NLTKTextSplitter(chunk_size=1000)
    return text_splitter.split_documents(docs)
```

Figure 12. Implementation of documents loading and splitting code.

3.2.3 Vectorizing Documents and Persisting Data

The process of documents vectorization and data persistence is shown in the figure below. First, the Cohere embedding model was used for documents vectorization. Before using this model, to the first visit is official website of Cohere to obtain the API Key. When using the Cohere embedding API, documents are uploaded to the Cohere server so vectors can be generated. The data is then transferred back and stored in the Chroma vector database running on the local computer. To install the Chroma vector database, execute the command: 'pip install langchain-chroma'. In addition, data persistence operations were performed to store the vectors persistently in the local 'vector_store' directory.

```
# Documents loading and splitting
split_docs = load_documents(path)

api_key = os.getenv('cohere_api_key')
# Defining vector embeddings
embeddings = CohereEmbeddings(cohere_api_key = api_key)

# Documents vectorize and persist data
docsearch = Chroma.from_documents(split_docs, embeddings, persist_directory=~"/vector_store")
docsearch.persist()
```

Figure 13. Implementation of code to vectorize documents and persist data.

3.2.4 Definition of Contextual Compression Retriever

A Contextual Compression Retriever is defined here for retrieving data from the vector database. Unlike general retrievers, it does not return the retrieved documents immediately, but compresses the retrieved documents and extracts relevant information and returns them. One challenge of retrieval is that when data is introduced into the system, we do not know what specific queries the data system will face. This means that the information most relevant to a query may be buried in a document with a lot of irrelevant text. So, some of the retrieved data may not be very relevant to the query. To solve this problem, the Contextual Compression Retriever compresses the retrieved documents so that only relevant information is returned, rather than immediately returning the retrieved documents as is.

```
# Defining text compressors and compression retrievers
llm = ChatCohere(cohere_api_key = api_key)
compressor = LLMChainExtractor.from_llm(llm)
compression_retriever = ContextualCompressionRetriever(
    base_compressor=compressor,
    base_retriever=docsearch.as_retriever(search_type="mmr")
)
```

Figure 14. Implementation of Contextual Compression Retriever code.

In Contextual Compression Retriever, a base retriever and a Document Compressor are required. The base retriever retrieves relevant document chunks from the original document and passes them to the Document Compressor for compression. In order to ensure that the information retrieved from the original document is relevant to the query while reducing the redundancy between the results, the base retriever uses the maximum marginal relevance search. The Document Compressor uses the LLMChainExtractor, which iterates the document chunks retrieved by the base retriever and extracts only the content related to the query from each document chunk to achieve a compression effect. The extracted information is finally given to the LLM, which will reduce the cost of LLM calls and improve the accuracy of LLM answers.

3.2.5 Defining Prompt Template

As shown in the figure below, the prompt word template defined in this thesis application. Prompt words are used to guide the model's response, helping the model understand the context and generate relevant content. In the prompt word template, "context" is used to place the extracted relevant information; and "question" is used to place the questions asked by the user. In addition, the template also contains specific instructions to guide the LLM to answer questions based on contextual information. In the case where the LLM cannot determine the answer, the template also clearly instructs it not to make up an answer, but to simply answer "I don't know."

```
# Define prompt template
template = """Use the following pieces of context to answer the question at the end.
If you don't know the answer, just say that you don't know, don't try to make up an answer.
Always say "thanks for asking!" at the end of the answer.

{context}

Question: {question}

Helpful Answer: """
QA_CHAIN_PROMPT = PromptTemplate.from_template(template)
```

Figure 15. Definition of prompt template code.

3.2.6 Defining ConversationBufferMemory

Because this thesis application needs to have a conversation with the LLM, the application needs to remember previous interaction information in order to reference information previously mentioned in the conversation. ConversationBufferMemory was used to manage and store the conversation history. ConversationBufferMemory is a memory module in Langchain, which is used to track conversation history. The code is shown in the figure below. ConversationBufferMemory is an extremely simple form of memory that just keeps a list of chat messages in a buffer and passes those into the prompt template. There are four parameters, 'memory_key' defines the memory variable 'chat_history' used to store historical chat records; 'input_key' and 'output_key' define 'question' and

'answer' to be saved to the chat information history respectively; 'return_messages' is set to 'true', indicating that the chat information is returned as a message list, which is very useful for application scenarios that need to have a conversation with the LLM.

```
# Define ConversationBufferMemory to manage and store conversation history
memory = ConversationBufferMemory(
    memory_key="chat_history",
    input_key='question',
    output_key='answer',
    return_messages=True
)
```

Figure 16. Definition of ConversationBufferMemory code.

3.2.7 Creating a ConversationalRetrievalChain Instance

The next step is to create a ConversationalRetrievalChain Instance. The code is shown in the figure below. ConversationalRetrievalChain is a chain class in LangChain. It can be used to have a conversation with documents. If there is previous conversation history, it uses an LLM to rewrite the conversation into a query to send to a retriever (otherwise it just uses the newest user input). It then fetches those documents and passes them (along with the conversation) to an LLM to respond.

```
# Creating a ConversationalRetrievalChain Instance
qa = ConversationalRetrievalChain.from_llm(
    llm,
    retriever=compression_retriever,
    memory=memory,
    return_source_documents=True,
    combine_docs_chain_kwargs={"prompt": QA_CHAIN_PROMPT}
)
```

Figure 17. Creating a ConversationalRetrievalChain instance code.

The 'from_llm' method of the ConversationalRetrievalChain is called to load the chain. This method makes use of the previously defined compression_retriever, memory, and prompt, and additionally specifies the llm parameter. This parameter specifies the default language model used in all stages of the chain, including

the question generation and answering process. In addition, the 'return_source_documents' property is set to 'True', which enables the retrieved source documents to be returned as part of the final output. Therefore, this setting makes the ConversationalRetrievalChain ready, waiting for the input question to generate the answer.

3.2.8 Verifying Folder Selection Before Processing User Queries

Before processing the user's question, the system first needs to verify whether the user has selected a folder on the front end. Only when the user selects a folder will the path of the folder be passed to the back end. After that, the system will load the files in the folder and perform processing steps such as file splitting and vectorization. Only after these steps are completed and the ConversationalRetrievalChain instance is successfully created, the system can process the user's query and interact. If the user has not selected a folder on the front end, the back end will return an error code '400'.

```
# Determine whether the user has selected a folder
if state["qa"] is None:
    raise HTTPException(status_code=400, detail="Documents not set. Please set the document path first.")
```

Figure 18. Implementation of code to determine whether the user has selected a folder.

3.2.9 Interacting with User Questions

Before answering the question asked by the user, the application must first determine whether the question is relevant to the document content. If the question is not relevant to the document, the system will return a prompt to the user stating that the question is not relevant to the document content. If the question is relevant to the document, the application calls the 'invoke' method of ConversationalRetrievalChain to generate the answer. After completing this process, the application returns the answer and its source to the front end for the user to view.

```

# Determine whether the user's question is related to the documents
if(check_relevance(question.question, 10000)):
    return {"answer": "" "Your question is not related to the local document library you selected.
        Please try asking a question related to the documentation library." ""}
else:
    result = state["qa"].invoke({"question": question.question})
    sources = []
    for source_document in result["source_documents"]:
        source = source_document.metadata["source"]
        filename = source.split('/')[-1]
        sources.append(filename)
    unique_sources = list(set(sources))
    # Return to answer and sources
    return {"answer": result['answer'], "unique_sources": unique_sources}

```

Figure 19. Implementation of Code for Interacting with User Queries.

3.2.10 Presentation of Answer and Sources

As shown in the figure below, the final display effect of the front-end interactive page not only presents the answer, but also includes the sources of the answer. This design enhances the credibility of the answer and provides users with a more transparent source of information.

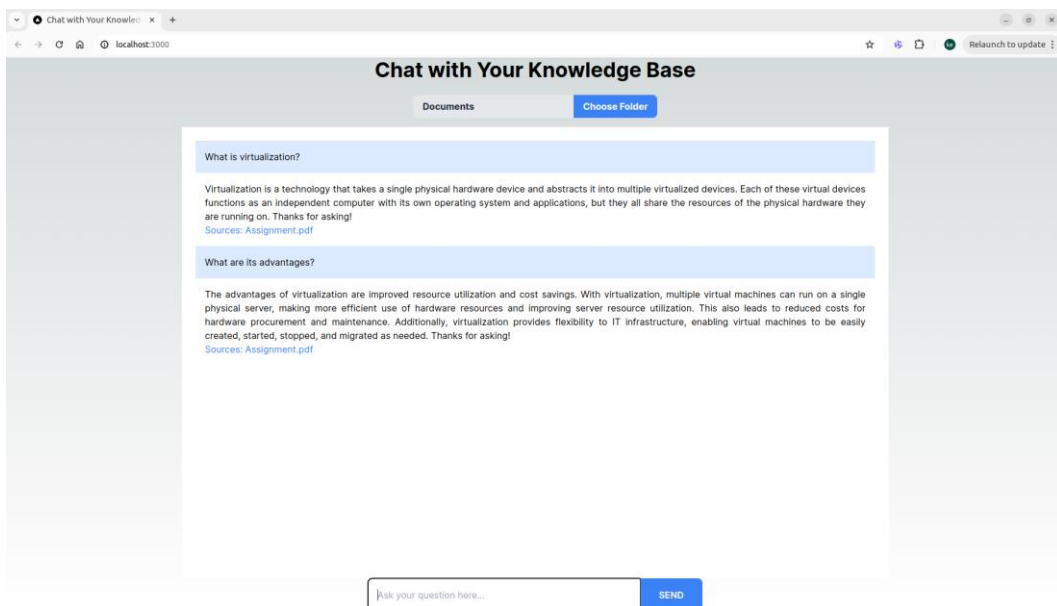


Figure 20. Final presentation of the front-end interaction page.

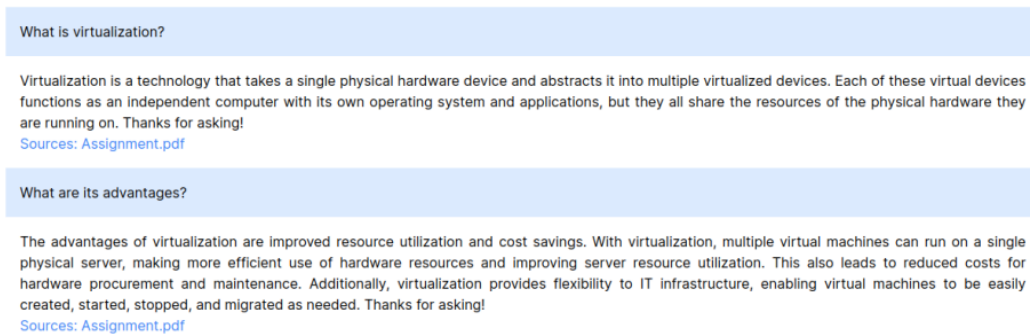


Figure 21. Screenshot of front-end page displaying answers and their sources.

3.3 Challenges Encountered

During the application development process, I mainly encountered the following three challenges.

3.3.1 Hardware Limitations in Local Deployment of LLM

In this study, the Llama 2 7B model was first deployed locally in order to implement localized LLM applications. However, due to limitations of hardware resources, this attempt failed to achieve the expected results. Specifically, despite processing only a small number of documents, the document vectorization process still took several minutes; similarly, there was a significant delay in question generation. This problem is mainly caused by the resource limitations of the local computer. The computer is only equipped with 16GB of RAM and no GPU. Under this hardware configuration, Llama 2 7B operates extremely inefficiently.

To explore possible solutions, there was an attempt to deploy several models with smaller parameter sizes, hoping to speed up processing by reducing the computational load. However, although these small models have improved in running speed, the quality of the answers they currently generate is not ideal. Therefore, a decision was made to abandon the local deployment solution and instead use a cloud hosted LLM which has more resources and can be called via an API. By using Cohere's API and taking advantage of its better hardware facilities, processing speeds are significantly increased. In addition, the LLM performance provided by

Cohere is better than that of small models, ensuring high quality of answers. Ultimately, this solution effectively solves application requirements under limited local resources.

3.3.2 Irrelevance Between User Queries and Document Contents

During application development, it is often challenging to handle the situation where the user query is not relevant to the content of the documents in the selected folder. In this case, LLM usually automatically generates an answer instead of pointing out the irrelevance of the query to the content of the file. To improve the usability of the application, an approach was adopted based on similarity data. This approach involves using a retriever to find content related to the user query in the documents, and at the same time obtain similarity data of each document fragment with the query. With this data, I can determine the relevance of the query to the document. If the query is not relevant to the document content, the application will directly prompt the user, as shown in the figure below; if the similarity is within the preset threshold, the query is considered relevant to the document and the processing continues.

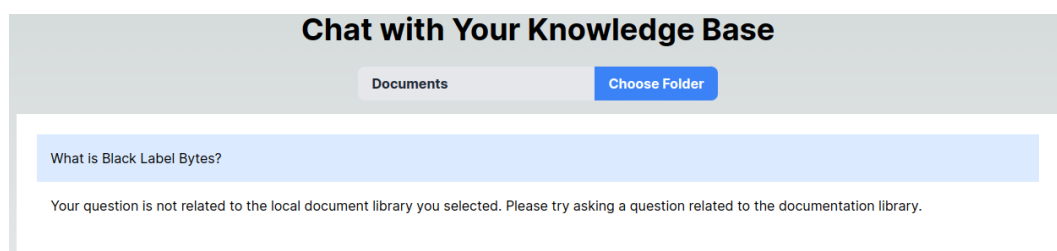


Figure 22. Example of answer when the user query is not relevant to the content of the documents in the selected folder.

However, this method has limitations when dealing with queries that need to be combined with historical messages. For example, for a question like "What are its benefits?", it is not directly relevant to the document and needs to be combined with historical information to provide a relevant answer. This type of question is handled in the "ConversationalRetrievalChain", and an effective method is not to be found to perfectly handle this type of situation. Therefore, the threshold was

set higher to filter out completely irrelevant queries as much as possible. Although this method is not perfect, it can at least exclude interference that is completely irrelevant to the content and ensure the accuracy and relevance of information retrieval.

3.3.3 How to Increase the Credibility of LLM Answers

For the answers generated by LLM, users may have such doubts: Are these answers generated based on the retrieved document chunks? This doubt is particularly common for users who attach great importance to the correctness of answers. To solve this problem, the solution is to display the source of the answer below the answer in the user interface, as shown in the figure below. The specific implementation is to set the 'return_source_documents' property to 'True' in the 'from_llm' method of ConversationalRetrievalChain, so that the retrieved source documents can be returned as part of the final output. These returned source documents include the file name of the document. Displaying these file names below the answer in the user interface allows users to clearly know the source of the answer. If users have doubts about an answer, they can also find and review the source document by file name, which increases the credibility of the answer.

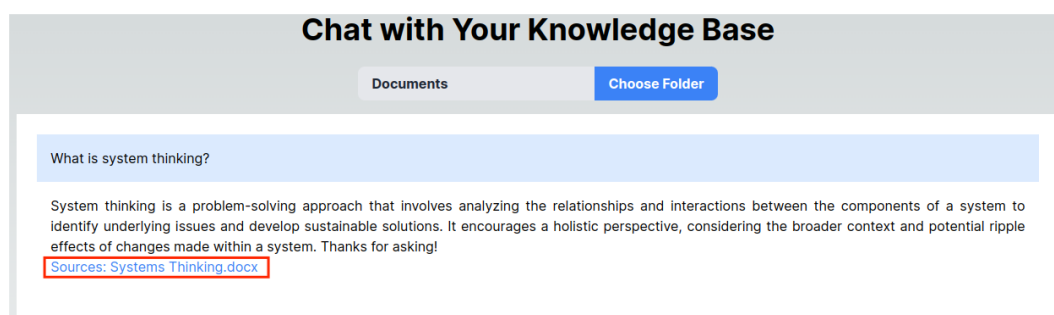


Figure 23. Example of answer source.

3.3.4 Vectorization Processing Challenges

As the number of documents increases, the vectorization process becomes very time-consuming. This is because after these documents are divided into fragments,

they will be uploaded to the Cohere server and vectorized by the Cohere embedding model. This process takes a lot of time, both in uploading the document and in the actual vectorization process itself. As the number of documents increases, the amount of data to be processed increases accordingly, resulting in longer upload and calculation times. Currently, this issue has not been resolved. For specific test results, see the performance analysis section below.

4 RESULTS AND ANALYSIS

In this section, we first review the research question and then present the key functionalities implemented in the application. Furthermore, we analyze the performance of the application, explore ethical considerations of data usage, and evaluate potential limitations of the application.

The main goal of this thesis is to integrate RAG with LLM through LangChain to improve the efficiency and accuracy of file retrieval and real-time query answering in personal computer systems. The developed application allows users to interact with files stored on their computers through a conversational interface, leveraging the computational power of LLM to understand and generate contextually relevant responses based on the file content. This helps users quickly find relevant files and answer users' questions about the file in real time.

4.1 Function Implementation

The first major function implemented in this application is document vectorization. This process involves converting the text data in the document into a digital format that can be interpreted by the machine. First, we load the documents in the folders selected by the user. Then the content of the documents is segmented and vectorized. Each document is split into manageable chunks to maintain semantic coherence within the segments. Next, these document chunks are passed to the Cohere server by calling the Cohere API, converted into vectors using Cohere's embedding model, and the resulting vectors are stored in a local vector database. The vectorization process facilitates efficient retrieval.

The second major function is query processing and response generation, which is the core of the application. When a user enters a query, the application first vectorizes this query. It then performs a maximum marginal relevance search on the document vectors stored in the vector database to find relevant document chunks.

This retrieval process is enhanced by RAG technology, which ensures that the information used to generate the response is the most applicable to the query. The selected chunks are used as context for the LLM to generate coherent and contextually appropriate responses. This process not only leverages the powerful generative capabilities of the LLM, but also greatly reduces the possibility of the LLM generating inaccurate or irrelevant information.

Finally, we developed an interactive user interface to facilitate interaction between users and the application. FastAPI framework was used on the backend, and Next.js framework was used on the frontend to build a user-friendly conversational interface where users can ask questions and receive answers. The interface not only displays responses but also shows the source of the responses, which enhances the transparency and trust of the system.

These three core functions form the backbone of the application, and each plays a key role in ensuring the efficiency, accuracy, and usability of the system. Through the careful integration of these components, the application effectively leverages the capabilities of LLM and RAG to achieve efficient document retrieval and real-time query answering.

4.2 Performance Analysis

The performance of the application is analyzed from the following aspects: document vectorization efficiency, retrieval efficiency, average query response time, accuracy, and usability.

The application's document vectorization efficiency, retrieval efficiency, and average query response time are key metrics for measuring its performance. I tested the application's average documents vectorization time, average retrieval and response generation time using 10, 20, and 50 documents on a PC with only 16GB RAM and no GPU. The test results are detailed in Table 1.

Table 1. Test Results for Documents Vectorization, Retrieval and Response Generation Time.

Number of Documents	Documents Vectorization Time (seconds)	Retrieval and Response Generation Time (seconds)
10	24.88	13.85
20	90.47	11.04
50	313.56	8.74

According to the test results, the efficiency of this application still needs to be improved. Especially when the number of documents is large, document vectorization takes a long time. In addition, the average time to retrieve and generate a reply is about 10 seconds, which is not ideal. This may be related to the low configuration of the computer, and calling the LLM API may also increase the response delay. However, manually locating files and the specific information within them on a local computer often takes tens of seconds. If there are many files and the location is not clear, it may take several minutes to find the files. Therefore, this application still has some advantages in comparison.

In terms of accuracy, the application shows a considerable ability to understand and respond to user queries. This is mainly due to the integration of RAG technology, which significantly enhances the LLM's ability to generate precise answers by providing the LLM with context-sensitive chunks of documents. As shown in Figure 24 and Figure 25, the application can give an accurate answer to a question that requires an accurate explanation or the retrieval of specific data. An LLM without integrated RAG technology may give an ambiguous or fabricated answer.

Please explain RAG

RAG, or Retrieval-Augmented Generation, is an emerging field in AI that enhances language models' generation capabilities by incorporating external knowledge sources. The goal is to improve the accuracy and context-awareness of language models by providing them with additional information retrieved from databases or text corpora. The RAG ecosystem is expanding, with a growing number of AI applications and tools being developed to support its functionality. This includes research into multi-modal RAG, which aims to address complex problems by incorporating multiple types of data, such as text and images, and super-long contexts that can provide more comprehensive answers. Thanks for asking!

Sources: [Retrieval-Augmented Generation for Large Language Models.pdf](#), [Enterprise Data-Based LLM Application Architecture.pdf](#)

How does it work?

RAG, or Retrieval-Augmented Generation, is a process that involves three main parts: indexing, retrieval, and generation. In the indexing phase, documents or text snippets are indexed to create a database. During retrieval, relevant documents are retrieved from this database using a query. Finally, in the generation phase, a model generates a response based on the retrieved documents. There are two main approaches to RAG: RAG-Sequence and RAG-Token. In the RAG-Sequence approach, the model uses the same retrieved document to predict each target token. On the other hand, RAG-Token can predict each target token based on a different retrieved document, allowing for more flexibility in generating responses. Advanced RAG builds upon this by introducing optimization strategies around pre-retrieval and post-retrieval processes, aiming to enhance the quality of retrieval and improve the overall performance of the system. Thanks for asking!

Sources: [Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.pdf](#), [Retrieval-Augmented Generation for Large Language Models.pdf](#)

Figure 24. Example 1: questions requiring precise explanation.

What are the advantages of RAG compared with traditional information retrieval methods?

RAG, or Retrieval-Augmented Generation, offers several advantages over traditional information retrieval methods. One of its key strengths lies in its ability to index and retrieve information from a diverse range of data formats, including PDF, HTML, Word, and Markdown. By converting raw data into a uniform plain text format, RAG ensures consistency and facilitates efficient indexing. Another advantage of RAG is its use of fine-grained segmentation, where text is divided into smaller, contextually relevant chunks. This segmentation addresses the context limitations of language models, making the information more digestible and enabling more precise retrieval. Additionally, RAG employs a sliding window approach, which further enhances the retrieval process. This technique involves considering not just individual words but also their surrounding context, capturing the semantic relationships between words and improving the accuracy of information retrieval. The incorporation of metadata in the indexing process is another benefit of RAG. Metadata provides additional context and descriptive information about the data, allowing for more nuanced searches and facilitating a more comprehensive understanding of the retrieved content. Furthermore, RAG utilizes optimization methods to streamline retrieval. This includes employing embedding models to encode chunks of text into vector representations, enabling efficient similarity searches during retrieval. This optimization enhances the speed and accuracy of information retrieval, making the process more efficient overall. Thanks for asking!

Sources: [Retrieval-Augmented Generation for Large Language Models.pdf](#), [Enterprise Data-Based LLM Application Architecture.pdf](#)

Figure 25. Example 2: questions requiring precise explanation.

Usability was another important component of our performance analysis. Our design used a simple, intuitive conversational interface. Users only need to select a folder on the page to ask questions and receive corresponding information in natural language, with little to no additional training required. In addition, the page also displayed the source of the answer to enhance information transparency. This helped build trust and confidence among users, thereby increasing their reliance on the answers.

Overall, the performance analysis shows that the application needs improvement in terms of performance. It can meet the basic needs of users in terms of accuracy and usability, helping users to increase productivity. Furthermore, these results suggest that combining LLM and RAG technologies is beneficial for achieving greater potential for personal intelligent assistant applications.

4.3 The Ethics

The application was designed and developed with ethical considerations from the perspective of user privacy and transparency.

User privacy is a core ethical issue in the design and development of this application. User personal files may contain highly private information, so users are often reluctant to upload them to the cloud. It was originally planned to deploy the LLM locally, but due to hardware limitations, it was eventually adjusted to use the Cohere API. During the vectorization process of the documents, the application will upload the segmented document chunks to the Cohere server for processing, but will not upload the complete source documents. Although this method reduces the user's privacy protection level to a certain extent, Cohere attaches great importance to data privacy and has designed multiple mechanisms to ensure data security. In addition, this is explained in the user interface and reminded users not to upload private information (Figure 26). Therefore, this solution can be considered as a compromise solution. In the future, after hardware upgrade, deploying the LLM locally can be considered to further enhance user privacy protection.

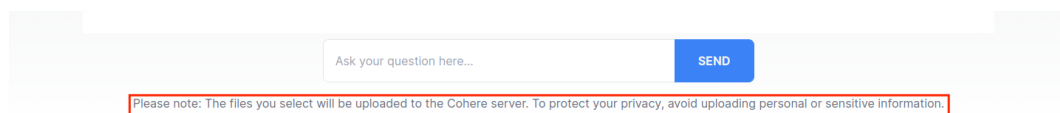


Figure 26. Remind users on the user interface not to upload private information.

Transparency is essential to building trust with users and ensuring the ethical operation of the application. The application clearly indicates the source of data used to generate responses, allowing users to verify the information themselves. By providing this level of transparency, the application enables users to make informed decisions based on accurate data. In cases where the application cannot provide a reliable response, it clearly states that it does not know, rather than fabricating answers, thereby preventing the spread of incorrect or misleading information.

Ethical considerations in the design of the application are carefully integrated to ensure privacy and transparency, creating a trustworthy environment for users to interact with confidence. In the future, as challenges in AI ethics arise, this application can continue to adapt and evolve.

4.4 Limitations

The application faced limitations in scalability, maintenance and updates, and consolidation of files from multiple storage locations.

Scalability is a significant challenge facing this application. The application's architecture is designed for individual users on personal computers, which means it relies primarily on local processing resources. This limits the ability of the application to efficiently process large-scale data or support multiple concurrent users. As the number of user files increases, it becomes a challenge to quickly and accurately retrieve relevant information from large file repositories. Implementing distributed computing or a hybrid cloud-local approach could alleviate this problem by balancing resource utilization and maintaining privacy, but this would require a complete redesign of the current system architecture.

Maintenance and updates are another limitation. Currently, LLM technology is updated and iterated quickly, and applications may need to be constantly adjusted. Because different LLM APIs are called, the code is different and needs to be rewritten. This may be a big challenge for users who are not very tech-savvy. If updates are ignored, the application may not run optimally, thus affecting the user experience.

Integrating files from multiple storage locations is also a challenge for the application. Users' files are not only stored on their local computers, but may also be stored in the cloud or other platforms, such as Google Drive. Currently, the application can only retrieve and process files stored on the local computer. Therefore,

future research should focus on how to integrate files from multiple storage locations to achieve comprehensive retrieval and answering capabilities.

While the application effectively achieves its primary goal of enhancing personal document retrieval and query answering, its current limitations highlight that there is much room for future improvement. Addressing scalability, update management issues, and consolidating documents from multiple storage locations will make the system more resilient and adaptable to different user needs.

5 CONCLUSIONS

In this section, a final summary of this thesis is given, reflecting on its methods, results and applicability, and finally exploring future research directions.

5.1 Conclusions

This thesis integrates RAG and LLM through the LangChain framework, proving that this method can effectively improve the efficiency of document retrieval. By first vectorizing the documents and then using the maximum marginal relevance search based on the user's query, fast and accurate retrieval of relevant document chunks is achieved, ensuring the accuracy and relevance of information retrieval.

In addition, the integration of RAG and LLM improves the accuracy of LLM's answers and reduces the possibility of hallucination. With this approach, an LLM is able to use the retrieved document chunks as context to provide a basis for generating relevant and precise answers, significantly optimizing query responses and ensuring the coherence and reliability of answers.

Although the application still has some shortcomings, it has basically achieved the goals desired. The effective integration of RAG and LLM not only improves the efficiency and accuracy of document retrieval in personal computer file systems, but also can answer users' queries about file content in real time. This study ultimately achieved the goal of developing a personalized intelligent assistant. The assistant can provide accurate and context-aware responses. This provides a paradigm for the development of smarter and more efficient personal intelligent assistants in the future.

5.2 Reflections

This thesis uses RAG technology to split larger documents into semantically related chunks and vectorize these chunks and store them in a vector database. Through maximum marginal relevance search, document chunks related to the user's query

are retrieved from the database and used together with the query as the context for LLM to generate answers. This method ensures that the answers generated by an LLM can fully meet the user's query needs and improves the accuracy of LLM replies.

In terms of applicability, the architecture combining RAG and LLM demonstrates its strong applicability in extracting personalized knowledge from users' local data. By leveraging user-specific file resources, the application can meet users' unique query needs, quickly generate relevant and accurate answers, and improve work and learning efficiency. This approach enables intelligent assistants to provide customized information services based on user needs, bringing significant benefits to individuals and businesses.

To ensure the reliability and transparency of the application, the application returns the source of the file chunks referenced by the answer to the front-end page display together with the answer, ensuring that users can verify the source of the answer at any time. This approach not only enhances the transparency of information, but also makes users more confident in the system's response. At the same time, the system clearly instructs LLM not to fabricate information when it is unable to determine the answer, thereby preventing the spread of false information.

5.3 Future Work

As discussed in Section 4.5 Limitations, there are still some areas that need improvement in the application. First, in order to enable the system to cope with larger-scale data and support multi-user concurrent queries, future work should focus on exploring the possibility of distributed architecture or hybrid cloud-local architecture. By adopting distributed computing or cloud storage, the system can effectively balance resource utilization while ensuring the privacy and security of data processing. Implementing this new architecture will require redesigning the

existing system to fully utilize the advantages of the distributed environment and improve the efficiency of document retrieval and answer generation.

In addition, the current LLM technology iteration speed is extremely fast, and applications face challenges in maintenance and upgrade. In order to ensure that applications can always provide the best performance, future work should integrate the main LLMs on the market into the system, so that users can easily switch after the release of new high-performance LLMs without the need for additional development programs. In addition, by improving documentation and user guides, users can easily modify configurations to ensure that the system always remains advanced and enhance user experience.

The current application mainly processes files stored on local computers, but future work should expand the system to enable integration with external platforms, such as cloud storage services and enterprise-level databases. By integrating data from multiple storage locations, the system can achieve more comprehensive retrieval and answering capabilities, providing users with deeper and more diverse ways to acquire knowledge. This functional expansion will help create a more comprehensive and flexible personalized intelligent assistant.

REFERENCES

- Amazon Web Services. (n.d.). What is LangChain? Retrieved 2024-03-24 from <https://aws.amazon.com/what-is/langchain/>
- Bahdanau, D., Cho, K., & Bengio, Y. (2014). Neural Machine Translation by Jointly Learning to Align and Translate. Retrieved 2024-03-11 from <https://arxiv.org/abs/1409.0473>
- Bowman, S. R. (2023). Eight Things to Know about Large Language Models. Retrieved 2024-03-10 from <https://arxiv.org/abs/2304.00612>
- Chen, W., He, W., Shen, J., Tian, X., & Wang, X. (2023). Systematic analysis of artificial intelligence in the era of industry 4.0. Retrieved 2024-03-07 from <https://www.tandfonline.com/doi/full/10.1080/23270012.2023.2180676>
- Cohere. (2024a). Semantic Search. Retrieved 2024-03-23 from <https://docs.cohere.com/docs/semantic-search>
- Cohere. (2024b). Embed. Retrieved 2024-05-12 from <https://cohere.com/embed>
- FastAPI. (n.d.). FastAPI Documentation. Retrieved 2024-03-26 from <https://fastapi.tiangolo.com/>
- Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Wang, M., & Wang, H. (2023). Retrieval-Augmented Generation for Large Language Models: A Survey. Retrieved 2024-03-23 from <https://arxiv.org/abs/2312.10997>
- Giray, L. (2023). Prompt Engineering with ChatGPT: A Guide for Academic Writers. Retrieved 2024-03-16 from <https://link.springer.com/article/10.1007/s10439-023-03272-4>
- Han, Y., Liu, C., & Wang, P. (2023). A Comprehensive Survey on Vector Database: Storage and Retrieval Technique, Challenge. Retrieved 2024-03-24 from <https://arxiv.org/abs/2310.11703>
- LangChain. (n.d.a). Q&A with RAG. Retrieved 2024-03-23 from https://python.langchain.com/v0.1/docs/use_cases/question_answering/
- LangChain. (n.d.b). Retrieval. Retrieved 2024-03-24 from https://python.langchain.com/v0.1/docs/modules/data_connection/

- LangChain. (n.d.c). Vector stores. Retrieved 2024-03-24 from https://python.langchain.com/docs/modules/data_connection/vectorstores/
- LangChain. (n.d.d). LangChain Introduction. Retrieved 2024-03-25 from https://python.langchain.com/docs/get_started/introduction/
- LangChain. (n.d.e). LangChain Prompts. Retrieved 2024-03-25 from https://python.langchain.com/docs/modules/model_io/prompts/
- LangChain. (n.d.f). LangChain Memory. Retrieved 2024-03-25 from <https://python.langchain.com/docs/modules/memory/>
- LangChain. (n.d.g). Split by tokens. Retrieved 2024-03-25 from https://python.langchain.com/v0.1/docs/modules/data_connection/document_transformers/split_by_token/
- Liu, G., & Yang, R. (2023). How much computing power does ChatGPT require. Retrieved 2024-05-07 from <https://vip.nxny.com/online/viewer.asp?rnd=230213084024315>
- Neelakantan, A., Weng, L., Power, B., & Jang, J. (2022). Introducing text and code embeddings. Retrieved 2024-03-23 from <https://openai.com/blog/introducing-text-and-code-embeddings>
- Paschen, U., Pitt, C., & Kietzmann, J. (2020). Artificial intelligence: Building blocks and an innovation typology. Retrieved 2024-03-07 from <https://www.sciencedirect.com/science/article/pii/S000768131930151X>
- Sharma, A., & Kumar, S. (2023). Ontology-based semantic retrieval of documents using Word2vec model. Retrieved 2024-03-07 from <https://www.sciencedirect.com/science/article/pii/S0169023X2200101X>
- Stöffelbauer, A. (2023). How Large Language Models work. Retrieved 2024-03-11 from <https://medium.com/data-science-at-microsoft/how-large-language-models-work-91c362f5b78f>
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention Is All You Need. Retrieved 2024-03-13 from <https://arxiv.org/abs/1706.03762>

White, J., Fu, Q., Hays, S., Sandborn, M., Olea, C., Gilbert, H., Elnashar, A., Spencer-Smith, J., & Schmidt, D. C. (2023). A Prompt Pattern Catalog to Enhance Prompt Engineering with ChatGPT. Retrieved 2024-03-16 from <https://arxiv.org/abs/2302.11382>

Wikipedia. (2024). Large language model. Retrieved 2024-03-10 from https://en.wikipedia.org/wiki/Large_language_model

XQ. (2024a). Explained: Attention Mechanism in AI. Retrieved 2024-03-11 from <https://medium.com/the-research-nest/explained-attention-mechanism-in-ai-e9bb6f0b0b4d>

XQ. (2024b). Explained: Transformers for Everyone. Retrieved 2024-03-11 from <https://medium.com/the-research-nest/explained-transformers-for-everyone-af01cbe600c5>

Ye, H., Liu, T., Zhang, A., Hua, W., & Jia, W. (2023). Cognitive Mirage: A Review of Hallucinations in Large Language Models. Retrieved 2024-03-07 from <https://arxiv.org/abs/2309.06794v1>